# Storage-Compute Separation: The Amazon Aurora Paradigm

Architectural decomposition based on the engineering lecture by **Arpit Bhayani** (*Asli Engineering*, Video ID: `DA5W8tO_7Nw`) and the landmark ACM SIGMOD research paper (*Verbitski et al., Amazon Web Services*).

> **The Foundational Axiom: "The Log is the Database"**
> In traditional monolithic relational databases, the bottleneck in the cloud is neither CPU compute nor disk I/O operations per second (IOPS)—**it is network I/O packet amplification**. Amazon Aurora fundamentally reshaped cloud database design by decoupling compute instances from storage nodes and establishing a radical principle: *the database engine only writes redo log records over the network; the storage tier itself materializes database data pages asynchronously*.

## 1. The Monolithic Bottleneck: Why Classical RDBMS Fails in the Cloud

In legacy architectures (e.g., standard MySQL running on AWS EC2 with mirrored EBS volumes across multiple Availability Zones), the database engine tightly couples SQL parsing, query optimization, buffer cache management, transaction isolation, and storage management into a single process.

**The Monolithic Cloud Failure Mode** • Network I/O Amplification

- **Heavy Page Writes:** Flushes full 16 KB dirty pages to disk, even if only a 10-byte column changed.
- **Doublewrite Buffer:** Writes pages twice to avoid partial/torn page corruption during crashes.
- **Synchronous Replication Multipliers:** Redo log, binary log (binlog), and data pages are replicated across AZ network boundaries independently, saturating network queues.

**Storage-Compute Separation Strategy** • Smart Storage Offloading

- **Stateless Compute:** SQL parsing, optimization, and buffer cache remain on compute instances.
- **Log-Only Network Traffic:** Compute sends small incremental redo log records; zero dirty page flushes.
- **Database-Aware Storage Fleet:** Multi-tenant storage nodes understand the redo log format and continuously apply changes to disk blocks independently.

## 2. Architectural Comparison: Monolithic RDBMS vs. Disaggregated Aurora

| Dimension | Traditional Engine (MySQL + EBS Replica) | Decoupled Engine (Amazon Aurora) | Architectural Impact |
|---|---|---|---|
| **Network Payload** | Redo log + binlog + doublewrite + dirty data pages (16 KB each). | Append-only redo log records (~100 to 200 bytes). Zero data pages. | Reduces network traffic by **up to 9x**, removing network transit bottlenecks. |
| **Durability & Quorum** | 2 AZ synchronous/asynchronous mirror replication (EBS level). | 6 storage replicas across 3 AZs with 4/6 write quorum and 3/6 read quorum. | Tolerates loss of an entire AZ plus an additional storage node without losing write availability. |
| **Crash Recovery** | Cold replay of WAL redo logs from the last checkpoint across all tables. | Instantaneous startup; redo replay is pushed down to background storage threads. | Near-zero crash recovery time; no waiting for checkpoint roll-forward. |

## 3. The Offloading Shift: Moving Redo Processing Down to Storage

The core conceptual breakthrough explained by Arpit Bhayani is shifting the line of abstraction. In traditional systems, storage is dumb block storage (EBS/SAN). In Aurora, storage is an intelligent, database-aware actor:

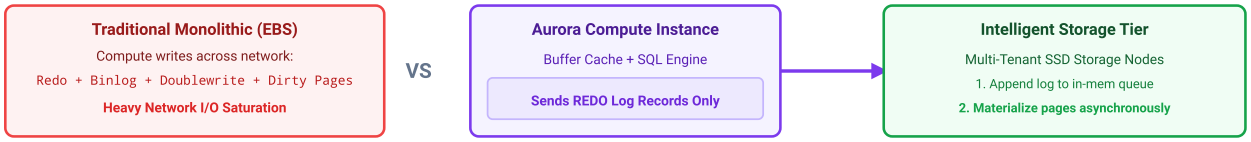



Figure 1: Architectural shift from noisy monolithic page flushing to compute log-pushing and intelligent storage materialization.

**The Compute Node Duties**

- Parses incoming SQL and runs query execution plans.
- Retains the in-memory **Buffer Cache** for zero-IO reads.
- Generates ordered Log Sequence Numbers (LSNs).
- Emits *only* redo log deltas to the storage fleet via concurrent UDP/TCP, avoiding expensive synchronous double-page writes.

**The Storage Node Background Pipeline**

- **Step 1 (Fast Path):** Append redo record to memory buffer + disk; send immediate ACK to compute.
- **Step 2 (Async):** Coalesce log records into on-disk pages.
- **Step 3 (Async):** Perform continuous peer-to-peer gossip repair.
- **Step 4 (Async):** Stream incremental backups directly to Amazon S3.

## 4. Asynchronous Page Materialization & Read Path Mechanics

How does read processing work if data pages are not explicitly flushed by compute?

**Buffer Cache Hits vs. Storage Page Reconstruction**

**Cache Hit (In-Memory):** If the requested page resides in the compute node's local buffer cache, it is served immediately in RAM with zero storage interaction.
**Cache Miss (Storage Fetch):** If the page is evicted, compute requests the page from storage at a specific LSN. The storage node takes its latest base snapshot page on SSD, replays any newer redo log records up to that requested LSN, and returns the fully reconstituted 16 KB data page.

## 5. Quorum Durability and Crash Recovery Without Log Replay

Separating compute and storage allows durability to be solved at the storage layer via quorum consensus. Aurora partitions the database volume into 10 GB **Protection Groups (PGs)**, each replicated 6 times across 3 distinct Availability Zones (2 copies per AZ).

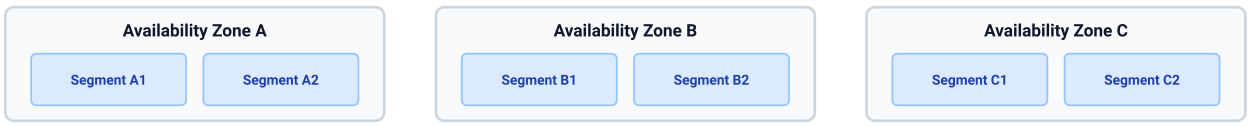



Figure 2: Aurora's 6-way storage allocation: write quorum Vw = 4/6, read quorum Vr = 3/6 across three distinct AZs.

**The 4/6 Write Quorum Guarantee**

Writing requires acknowledgments from 4 out of 6 storage copies. This ensures the system can withstand the **complete catastrophic loss of an entire AZ (2 copies) PLUS one additional node failure elsewhere (1 copy)** without dropping committed writes.

**Instantaneous Crash Recovery**

Traditional engines freeze during recovery while replaying hours of WAL logs from the last checkpoint. In Aurora, redo processing happens continuously in the storage nodes. When compute crashes, a new instance boots up and is ready immediately.

## 6. Architectural Industry Impact & Engineering Synthesis

**The Broader Cloud Revolution: Snowflake, AlloyDB, and Modern Warehouses**

The storage-compute separation pioneered by AWS Aurora sparked an industry-wide paradigm shift:

1. **Independent Elastic Scaling:** Compute instances can scale up or down (or pause entirely to zero) in seconds without migrating terabytes of underlying data disks.
2. **Zero-Lag Read Replicas:** Up to 15 read replicas can mount the identical shared storage tier with minimal replica lag (typically sub-10ms), eliminating physical data replication.
3. **Industry Standardization:** Cloud data warehouses (Snowflake, BigQuery) and modern OLTP databases (Google AlloyDB, Neon Serverless Postgres) all adopt variants of this disaggregated log architecture.